Sprint 5 : Functions, Scope, and Functional Thinking.

Introduction to Sprint 5

Concept Explanation (Sinhala): Sprint 5 එකෙන් අපි ප්‍රධාන වශයෙන්ම ඉගෙන ගන්නේ ආයෙ ආයෙත් පාවිච්චි කරන්න පුළුවන් (reusable) JavaScript code ලියන්නේ කොහොමද කියන එකයි. මීට කලින් sprints වලදී ඔයා values, variables, data types, strings, numbers, booleans, operators සහ conditions වගේ මූලික දේවල් ඉගෙන ගත්තා. දැන් අපි මේ ඔක්කොම logic එකතු කරලා functions ඇතුළට දාන්නයි යන්නේ. Function එකක් ලීවම අපිට එකම code එක එක සැරයක් ලියලා, ඕනෑම වාර ගණනක් ආයෙ ආයෙත් පාවිච්චි කරන්න පුළුවන් වෙනවා. අපේ ප්‍රධානම ඉලක්කය තමයි reusable code ලියන එක සහ අපේ code එකේ හැසිරීම කලින්ම නිවැරදිව හිතාගන්න පුළුවන් මට්ටමට (predictable) තියාගන්න එක.

Part A: The Core Idea

1. What Is a Function?

Concept Explanation (Sinhala): සරලවම කිව්වොත්, function එකක් කියන්නේ අපිට සේව් කරලා, නමක් දීලා, පස්සේ වෙලාවක run කරන්න පුළුවන් reusable code block එකකටයි. ඔයාට ඕනෑම වෙලාවක මේ code එක ආයෙ පාවිච්චි කරන්න පුළුවන්.

Code Block:

```
function sayHello() {
  return "Hello";
}

console.log(sayHello());
```

Code Breakdown (Sinhala):

1. `function sayHello()` කියන පේළියෙන් අපි `sayHello` කියලා නමක් ඇති function එකක් හදනවා (declare කරනවා).
2. `{ }` (curly braces) ඇතුළේ තියෙන්නේ මේ function එක run වෙනකොට ක්‍රියාත්මක විය යුතු code එකයි.
3. `return "Hello";` එකෙන් කරන්නේ "Hello" කියන string එක function එකෙන් පිටතට (output එකක් විදිහට) ලබා දෙන එකයි.
4. `sayHello()` කියලා ලීවම function එක run වෙනවා (call / invoke වෙනවා).
5. `console.log(sayHello());` එකෙන් ඒ ලැබෙන "Hello" කියන output එක screen එකේ print කරනවා.

2. Why Functions Are Useful

Concept Explanation (Sinhala): Functions පාවිච්චි කරන්නේ එකම code එක ආයෙ ආයෙ ලියමින් කාලය නාස්ති කරන එක (code repetition) නතර කරන්නයි.

Code Block: නරක ක්‍රමය *(Without functions):*

```
const price1 = 10;
const quantity1 = 3;
const total1 = price1 * quantity1;
console.log(total1);

const price2 = 5;
const quantity2 = 4;
const total2 = price2 * quantity2;
console.log(total2);
```

හොඳ ක්‍රමය *(With functions):*

```
function getTotal(price, quantity) {
  return price * quantity;
}

console.log(getTotal(10, 3));
console.log(getTotal(5, 4));
```

Code Breakdown (Sinhala):

- පළමු ක්‍රමයේදී අපිට අලුත් බඩු වර්ගයක් ලැබෙන හැම සැරේකම ගුණ කිරීමේ logic එක අලුතින්ම ලියන්න සිද්ධ වුණා.
- දෙවැනි ක්‍රමයේදී අපි `getTotal` කියලා එක function එකක් හැදුවා. ඒකට ඕනෑම price එකක් සහ quantity එකක් parameters විදිහට දීලා එකම logic එකෙන් ගාණ හදාගන්න පුළුවන් වුණා.

3. Function Mental Model

Concept Explanation (Sinhala): හිතන්න function එකක් කියන්නේ පොඩි මැෂින් එකක් කියලා. අපි ඒ මැෂින් එක ඇතුළට input දෙනවා, මැෂින් එක ඇතුළේ වැඩ ටික කෙරෙනවා, ඊටපස්සේ අපිට output එකක් ලැබෙනවා.

Code Block:

```
// Input: 10 සහ 3 -> [getTotal Machine] -> Output: 30

function getTotal(price, quantity) {
  return price * quantity;
}

console.log(getTotal(10, 3));
```

Code Breakdown (Sinhala):

- අපි `getTotal` කියන මැෂින් එකට input විදිහට `10` සහ `3` දෙනවා.
- මැෂින් එක ඇතුළේදී ඒ දෙක ගුණ වෙනවා (`price * quantity`).
- අවසානයේදී `30` කියන output එක `return` එක හරහා එළියට ලැබෙනවා.

Part B: Function Declaration and Invocation

4. Function Declaration

Concept Explanation (Sinhala): Function declaration එකක් කියන්නේ නමක් සහිතව අලුතින්ම function එකක් නිර්මාණය කරලා අර්ථ දක්වන එකටයි. හැබැයි මතක තියාගන්න, function එකක් declare කරපු පලියට ඒක run වෙන්නේ නැහැ.

Code Block:

```
function greet() {
  return "Hello";
}
```

Code Breakdown (Sinhala):

- මෙතනදී `function` කියන keyword එක පාවිච්චි කරලා, `greet` කියන නමින් function එකක් නිර්මාණය කරලා තියෙනවා.
- මේකෙන් කරන්නේ function එක ලෑස්ති කරලා තියන එක විතරයි. මේ code එක තවම run වෙලා නැහැ.

5. Function Invocation / Calling a Function

Concept Explanation (Sinhala): අපි හදපු function එකක් run කරන්න නම්, ඒකේ නම ලියලා අගට වරහන් `()` දෙකක් දාන්න ඕනේ. මේකට තමයි function එකක් "call කරනවා" නැත්නම් "invoke කරනවා" කියලා කියන්නේ.

Code Block:

```
function greet() {
  return "Hello";
}

greet(); // Function එක run වෙනවා
console.log(greet()); // Run වෙලා ලැබෙන ප්‍රතිඵලය print වෙනවා
```

Code Breakdown (Sinhala):

- `greet()` කියලා ලියපු සැනින් JavaScript එයාගේ මතකයේ තියන `greet` function එක ඇතුළට ගිහින් ඒක run කරනවා.
- `console.log(greet())` එකෙන් වෙන්නේ run වෙලා ලැබෙන "Hello" කියන අගය screen එකේ බලාගන්න සලස්වන එකයි.

6. Defining vs Calling

Concept Explanation (Sinhala): ආධුනිකයන් විදිහට මේ දෙකේ වෙනස හොඳින් තේරුම් ගන්න ඕනේ:

1. **Defining (හැදීම):** Function එකක් මෙහෙම වැඩ කරන්නේ කියලා ලියලා තියන එක විතරයි. (කිසිම දෙයක් වෙන්නේ නැහැ).
2. **Calling (ක්‍රියාත්මක කිරීම):** හදපු function එක ක්‍රියාත්මක කරලා වැඩක් ගන්න එක.

Rule: Function එකක් call කරනකම් ඒකෙන් කිසිම වැඩක් වෙන්නේ නැහැ!

Part C: Parameters and Arguments

7. What Is a Parameter?

Concept Explanation (Sinhala): Parameter එකක් කියන්නේ function එක ඇතුළේ පාවිච්චි කරන "තාවකාලික placeholder" එකකටයි. මේක ඇත්තම අගයක් නෙවෙයි, පස්සේ වෙලාවක ලැබෙන්න නියමිත අගයකට වෙන්කරපු හිස්තැනක් වගේ දෙයක්.

Code Block:

```
function greet(name) {
  return `Hello, ${name}`;
}
```

Code Breakdown (Sinhala):

- මෙතන වරහන් ඇතුළේ තියන `name` කියන්නේ parameter එකක්.
- මේක ඇතුළේ දැනට කිසිම ඇත්තම නමක් නැහැ, ඒක නිකන්ම හිස්තැනක් විතරයි.

8. What Is an Argument?

Concept Explanation (Sinhala): Argument එකක් කියන්නේ function එක ඇතුළට ඇත්තටම අපි පාස් කරන නියම වටිනාකම (real value) එකටයි. අපි function එක call කරනකොට තමයි මේ argument එක ලබා දෙන්නේ.

Code Block:

```
function greet(name) {
  return `Hello, ${name}`;
}

console.log(greet("Ada"));
```

Code Breakdown (Sinhala):

- මෙතනදී `"Ada"` කියන string එක තමයි argument එක වෙන්නේ.
- Function එක call වෙනකොට, `"Ada"` කියන argument එක, `name` කියන parameter එක වෙනුවට ආදේශ වෙනවා.
- ප්‍රතිඵලය විදිහට "Hello, Ada" ලැබෙනවා.

9. Parameter vs Argument

Concept Explanation (Sinhala): පහත වගුවෙන් මේ දෙකේ වෙනස ලෙහෙසියෙන්ම මතක තියාගන්න පුළුවන්:

වචනය	සරල තේරුම	උදාහරණය
Parameter	Function එක අර්ථ දක්වන කොට දාන placeholder එක	`name`
Argument	Function එක call කරනකොට දෙන ඇත්තම අගය	`"Ada"`

Code Block:

```
function add(a, b) {
  return a + b;
}

console.log(add(2, 3));
```

Code Breakdown (Sinhala):

- `a` සහ `b` කියන්නේ parameters.
- `2` සහ `3` කියන්නේ arguments.
- output එක `5` විදිහට ලැබෙනවා.

10. Multiple Parameters

Concept Explanation (Sinhala): Function එකකට එකකට වඩා වැඩි parameters ගණනක් තියාගන්න පුළුවන්. ඒ හැම එකක්ම කොමාවකින් (`,`) මඟින් වෙන් කරන්න ඕනේ. Arguments පාස් වෙනකොට පිළිවෙළටම පාස් වෙනවා (පළමු argument එක පළමු parameter එකටත්, දෙවැනි එක දෙවැනි එකටත් ආදී වශයෙන්).

Code Block:

```
function formatFullName(firstName, lastName) {
  return `${firstName} ${lastName}`;
}

console.log(formatFullName("Ada", "Lovelace"));
```

Code Breakdown (Sinhala):

- `firstName` එකට `"Ada"` ලැබෙනවා.
- `lastName` එකට `"Lovelace"` ලැබෙනවා.
- ප්‍රතිඵලය විදිහට "Ada Lovelace" ලැබෙනවා.

Part D: Return Values

11. What Does return Do?

Concept Explanation (Sinhala): `return` එකෙන් කරන්නේ function එක ඇතුළේ කරපු වැඩක අවසාන ප්‍රතිඵලය (answer එක) නැවත පිටතට ලබා දෙන එකයි.

Code Block:

```
function add(a, b) {
  return a + b;
}

const result = add(2, 3);
console.log(result);
```

Code Breakdown (Sinhala):

- `add(2, 3)` call වුණාම, function එක ඇතුළේ `2 + 3` එකතු වෙලා `5` හැදෙනවා.
- `return` නිසා ඒ `5` කියන අගය එළියට විසි වෙනවා.
- අපි ඒ එළියට ආපු `5` කියන අගය `result` කියන variable එකේ store කරගෙන පස්සේ print කරනවා.

12. return Stops the Function

Concept Explanation (Sinhala): JavaScript code එක run වෙනකොට `return` කියන keyword එක හමු වුණු සැනින්, function එක එතනින්ම සම්පූර්ණයෙන්ම නතර වෙනවා. ඊට පස්සේ තියන කිසිම code පේළියක් run වෙන්නේ නැහැ.

Code Block:

```
function checkAge(age) {
  if (age >= 18) {
    return "Adult";
  }
  return "Minor";
}

console.log(checkAge(20));
console.log(checkAge(15));
```

Code Breakdown (Sinhala):

- `checkAge(20)` call කරපු වෙලාවේ `age >= 18` සත්‍ය (true) වෙනවා. එතකොට `return "Adult";` කියන පේළිය run වෙනවා. Function එක එතනින්ම ඉවර වෙන නිසා, පල්ලෙහා තියන `return "Minor";` පේළියට යන්නේ නැහැ.
- `checkAge(15)` වලදී if condition එක අසත්‍ය (false) වෙන නිසා, පල්ලෙහා පේළියේ තියන `return "Minor";` run වෙනවා.

13. What Happens If There Is No return?

Concept Explanation (Sinhala): ඔයා function එකක් ඇතුළේ කිසිම දෙයක් `return` කරලා නැත්නම්, JavaScript ස්වයංක්‍රීයවම ඒ function එකෙන් `undefined` කියන value එක එළියට දානවා.

Code Block: වැරදි ක්‍රමය:

```
function sayHi() {
  const message = "Hi";
}

console.log(sayHi()); // Output: undefined
```

නිවැරදි ක්‍රමය:

```
function sayHi() {
  const message = "Hi";
  return message;
}

console.log(sayHi()); // Output: Hi
```

Code Breakdown (Sinhala):

- පළමු උදාහරණයේදී අපි variable එකක් හැදුවට ඒක එළියට ලබා දීලා නැහැ (no return statement). ඒ නිසා output එක `undefined` වෙනවා.
- දෙවැනි එකේදී `return message;` මඟින් නිවැරදිව අගය පිටතට ලබා දෙනවා.

Part E: return vs console.log()

14. console.log() Prints a Value

Concept Explanation (Sinhala): `console.log()` එකෙන් කරන්නේ අගයක් screen එකේ (browser console එකේ) නිකන්ම පෙන්වන එක විතරයි. ඒකෙන් ලැබෙන අගය අපිට වෙනත් ගණනය කිරීමකට ආයෙ පාවිච්චි කරන්න බැහැ.

Code Block:

```
function add(a, b) {
  console.log(a + b);
}

add(2, 3);
```

Code Breakdown (Sinhala):

- මෙතනදී screen එකේ `5` කියලා වැටෙනවා ඇත්ත. හැබැයි මේ function එකෙන් එළියට කිසිම අගයක් `return` වෙන්නේ නැහැ.

15. return Gives a Reusable Value

Concept Explanation (Sinhala): `return` එකෙන් අපිට නැවත වෙනත් වැඩකට පාවිච්චි කරන්න පුළුවන් සැබෑ අගයක් ලබා දෙනවා.

Code Block:

```
function add(a, b) {
  return a + b;
}

const result = add(2, 3);
const doubled = result * 2;
console.log(doubled); // Output: 10
```

Code Breakdown (Sinhala):

- `add(2, 3)` එකෙන් `5` කියන අගය `return` කළා.
- ඒ ලැබුණු `5` කියන අගය අපිට `doubled` කියන අලුත් ගණනය කිරීමට නැවත පාවිච්චි කරන්න පුළුවන් වුණා.

16. Beginner Rule for return and console.log()

Concept Explanation (Sinhala):

- **පාලනය:** Function එකකින් අගයක් එළියට ගන්න ඕනේ නම් හැමතිස්සෙම `return` පාවිච්චි කරන්න.
- **පරීක්ෂා කිරීම:** Code එක ලියන අතරතුර මොකක් හරි අගයක් ඇත්දෙකන් බලාගන්න ඕනේ නම් විතරක් `console.log()` පාවිච්චි කරන්න.

Part F: Default Parameters

17. What Is a Default Parameter?

Concept Explanation (Sinhala): Default parameter එකක් කියන්නේ fallback value (විකල්ප අගයක්) එකකටයි. අපි function එක call කරනකොට argument එකක් දුන්නේ නැත්නම්, JavaScript ස්වයංක්‍රීයවම මේ default අගය පාවිච්චි කරනවා.

Code Block:

```
function greet(name = "Guest") {
  return `Hello, ${name}`;
}

console.log(greet("Anna")); // Argument එකක් දුන්නා
console.log(greet());       // Argument එකක් දුන්නේ නැහැ
```

Code Breakdown (Sinhala):

- පළමු call එකේදී `"Anna"` කියන argument එක දුන්න නිසා "Hello, Anna!" කියලා ලැබෙනවා.
- දෙවැනි call එකේදී අපි කිසිම දෙයක් දුන්නේ නැහැ. ඒ නිසා `name` වලට default දාලා තිබුණු `"Guest"` කියන අගය අරගෙන "Hello, Guest!" විදිහට output එක දෙනවා.

18. Why Default Parameters Are Useful

Concept Explanation (Sinhala): Default parameters පාවිච්චි කිරීමෙන් අපේ code එක ආරක්ෂිත වෙනවා. එහෙම අනපේක්ෂිත `undefined` වැටීම් වළක්වා ගන්න පුළුවන්.

Code Block:

```
// Default නැතුව:
function greetWithoutDefault(name) {
  return `Hello, ${name}`;
}
console.log(greetWithoutDefault()); // Output: Hello, undefined!
```

Part G: Function Expressions and Anonymous Functions

19. Function Declaration vs Function Expression

Concept Explanation (Sinhala):

- **Function Declaration:** සාමාන්‍ය විදිහට නමක් දාලා කෙළින්ම function එකක් ලියන එක.
- **Function Expression:** Function එකක් සාමාන්‍ය value එකක් වගේ හදලා, ඒක variable එකක් ඇතුළේ store කරන එක.

Code Block:

```
// Function Declaration
function addDecl(a, b) {
  return a + b;
}

// Function Expression
const addExpr = function(a, b) {
  return a + b;
};

console.log(addDecl(2, 3));
console.log(addExpr(2, 3));
```

Code Breakdown (Sinhala):

- ක්‍රම දෙකෙන්ම අවසාන ප්‍රතිඵලය එක වුණත්, දෙවැනි ක්‍රමයේදී අපි function එක variable එකක් ඇතුළේ store කරලා තියෙනවා.

20. Anonymous Functions

Concept Explanation (Sinhala): නමක් නැති function එකකට අපි anonymous function එකක් කියලා කියනවා. මේවා සාමාන්‍යයෙන් ලියන්නේ තවත් variable එකකට assign කරන්න හරි, වෙනත් function එකකට argument එකක් විදිහට පාස් කරන්න හරි විතරයි.

Code Block:

```
const add = function(a, b) {
  return a + b;
};
```

Code Breakdown (Sinhala):

- මෙතන `function(a, b)` කියන කොටසට තනියම නමක් නැහැ. ඒ නිසා ඒක anonymous function එකක්. හැබැයි අපි ඒක `add` කියන variable එකේ store කරපු නිසා, `add(2, 3)` කියලා call කරන්න පුළුවන්.

Part H: Arrow Functions

21. What Is an Arrow Function?

Concept Explanation (Sinhala): Arrow function එකක් කියන්නේ සාමාන්‍ය function එකක් වඩාත් කෙටියෙන් ලියන්න පුළුවන් නවීන ක්‍රමයකටයි. මෙතනදී `function` කියන keyword එක වෙනුවට ඊතල සලකුණක් `=>` පාවිච්චි කරනවා.

Code Block:

```
// Normal Function
const addNormal = function(a, b) {
  return a + b;
};

// Arrow Function
const addArrow = (a, b) => {
  return a + b;
};
```

22. Arrow Function with Implicit Return

Concept Explanation (Sinhala): Arrow function එකක් ඇතුළේ එකම එක පේළියක code එකක් විතරක් තියනවා නම්, අපිට curly braces `{ }` සහ `return` කියන keyword එක අයින් කරලා තවත් කෙටි කරන්න පුළුවන්. මේකට implicit return කියලා කියනවා. JavaScript විසින්ම ස්වයංක්‍රීයව අගය return කරගන්නවා.

Code Block:

```
const add = (a, b) => a + b;

console.log(add(2, 3)); // Output: 5
```

23. Arrow Function Beginner Rule

Concept Explanation (Sinhala): මුලින්ම ඉගෙන ගන්නකොට සාමාන්‍ය `function` syntax එකෙන් ලියන්න පුරුදු වෙන්න. Return සහ parameters ගැන හොඳ අවබෝධයක් ලැබුණට පස්සේ කෙටි arrow functions පාවිච්චි කරන්න පටන් ගන්න. ඉක්මන් වෙන්න එපා!

Part I: Scope

24. What Is Scope?

Concept Explanation (Sinhala): Scope එකක් කියන්නේ code එකේ යම් කිසි variable එකක් පාවිච්චි කරන්න අවසර තියන සීමාවටයි (boundary). සීමාවෙන් පිටතදී ඒ variable එක පාවිච්චි කරන්න ගියොත් error එකක් ලැබෙනවා.

25. Global Scope

Concept Explanation (Sinhala): කිසිම function එකකට හෝ block එකකට පිටින්, මුළු file එකේම ඉහළින්ම හදන variables global scope එකටයි. මේවා මුළු code එකේ ඕනෑම තැනක ඉඳන් පාවිච්චි කරන්න පුළුවන්.

Code Block:

```
const appName = "Study App";

function showAppName() {
  return appName;
}

console.log(appName); // Output: Study App
console.log(showAppName()); // Output: Study App
```

Code Breakdown (Sinhala):

- `appName` කියන්නේ global variable එකක්. ඒ නිසා ඒක function එක ඇතුළෙත්, එළියෙත් දෙකේම වැඩ කරනවා. හැබැයි ඕනාවට වඩා global variables හදාගැනීමෙන් code එක අවුල් වෙන්න පුළුවන් (messy).

26. Local / Function Scope

Concept Explanation (Sinhala): Function එකක් ඇතුළේ හදන variables පාවිච්චි කරන්න පුළුවන් ඒ function එක ඇතුළේදී විතරයි. ඒවා function එකෙන් එළියට පේන්නේ නැහැ.

Code Block:

```
function createMessage() {
  const message = "Hello";
  return message;
}

console.log(createMessage()); // වැඩ කරනවා
console.log(message); // ERROR! ReferenceError
```

Code Breakdown (Sinhala):

- `message` කියන variable එක හදලා තියන්නේ `createMessage` function එක ඇතුළේ විතරයි. ඒ නිසා එළියේ ඉඳන් පාවිච්චි කරන්න ගියොත් JavaScript වලට ඒක හොයාගන්න බැරි වෙනවා (ReferenceError).

27. Block Scope

Concept Explanation (Sinhala): Curly braces `{ }` යුගලයක් ඇතුළත තියන සීමාවට block එකක් කියලා කියනවා (උදාහරණ: `if` conditions, `for` loops). `let` සහ `const` පාවිච්චි කරලා block එකක් ඇතුළේ හදන variables පාවිච්චි කරන්න පුළුවන් ඒ block එක ඇතුළේ විතරයි.

Code Block:

```
if (true) {
  const score = 90;
  console.log(score); // වැඩ කරනවා (90)
}

console.log(score); // ERROR! ReferenceError
```

28. Scope Beginner Rule

Concept Explanation (Sinhala): Variables හැමතිස්සෙම තියාගන්න ඕනේ ඒවට අවශ්‍ය කුඩාම scope එක ඇතුළේ විතරයි. අනවශ්‍ය විදිහට හැමදේම global කරන්න එපා.

Code Block: නරක ක්‍රමය:

```
let total; // අනවශ්‍ය ලෙස global කර ඇත
function calculateTotal(price, quantity) {
  total = price * quantity;
}
```

හොඳ ක්‍රමය:

```
function calculateTotal(price, quantity) {
  const total = price * quantity; // Local විතරක් ඇතුළේ තමයි
  return total;
}
```

Part J: Pure Functions

29. What Is a Pure Function?

Concept Explanation (Sinhala): Pure function එකක් කියන්නේ ප්‍රධාන නීති දෙකක් අනුගමන රකින function එකකටයි:

1. **එකම input එකට හැමදාම ලැබෙන්නේ එකම output එකයි:** වෙනසක් වෙන්නේ නැහැ.
2. **කිසිම side effect එකක් නැහැ:** (function එකෙන් පිටත තියන කිසිම variable එකක් වෙනස් කරන්නේ හෝ බලපෑම් කරන්නේ නැහැ).

Code Block:

```
function add(a, b) {
  return a + b;
}

console.log(add(2, 3)); // හැමදාම 5 ම ලැබෙයි!
console.log(add(2, 3)); // හැමදාම 5 ම ලැබෙයි!
```

30. Pure Function Example: Tax Calculation

Concept Explanation (Sinhala): Pure function එකක් වැඩ කරන්නේ එයාට ලැබෙන parameters පාවිච්චි කරලා විතරයි. එළියේ තියන රහස් දත්ත මත යැපෙන්නේ නැහැ.

Code Block:

```
function calculateTax(price, taxRate) {
  return price * taxRate;
}

console.log(calculateTax(100, 0.1)); // Output: 10
```

31. Why Pure Functions Are Easier to Test

Concept Explanation (Sinhala): Pure functions හරිම ලේසියෙන් ටෙස්ට් කරන්න පුළුවන් මොකද ඒවයේ හැසිරීම හැමවෙලාවෙම එක වගේ (predictable) නිසා. අපිට `console.assert` පාවිච්චි කරලා අපේක්ෂිත ප්‍රතිඵලය ලැබෙනවාද කියලා බලාගන්න පුළුවන්.

Code Block:

```
function isPassing(score) {
  return score >= 70;
}

console.assert(isPassing(85) === true, "85 should pass");
console.assert(isPassing(60) === false, "60 should not pass");
```

Part K: Side Effects

32. What Is a Side Effect?

Concept Explanation (Sinhala): Function එකක් ඇතුළේ ඉඳන් එතනින් එළියේ තියන දෙයක් වෙනස් කරනවා නම්, නැත්නම් වෙනත් බාහිර දෙයක් එක්ක සම්බන්ධ වෙනවා නම් අපි ඒකට side effect එකක් කියලා කියනවා.

පොදු side effects:

- Screen එකේ print කිරීම (`console.log()`)
- Global variables වෙනස් කිරීම
- Web page එකේ HTML වෙනස් කිරීම
- Alerts පෙන්වීම (`alert()`)
- Browser memory එකට දත්ත ලියීම

Code Block:

```
function showMessage(message) {
  console.log(message); // Side Effect එකක් සිදු කරයි
}
```

33. Side Effects Are Not Always Bad

Concept Explanation (Sinhala): Side effects කියන්නේ නරක දෙයක් නෙවෙයි, සැබෑ වෙබ් අඩවියක් වැඩ කරන්න නම් side effects අනිවාර්යයෙන්ම ඕනේ (උදාහරණ: screen එකේ text වෙනස් කරන්න, buttons click වලට ප්‍රතිචාර දක්වන්න). හැබැයි හොඳ පුරුද්දක් විදිහට, අපි ගණනය කිරීම් කරන කොටසයි (calculation) සහ side effects සිදුකරන කොටසයි වෙන් කරලා ලියන්න ඕනේ.

34. Separating Calculation from Side Effects

Concept Explanation (Sinhala): ගණනය කිරීම් කරන pure function එකක් වෙනම ලියලා, console.log වගේ side effects වෙනම පේළියක සිදු කිරීම code එක පිරිසිදු කරන හොඳ ක්‍රමයක්.

Code Block: අපිරිසිදු ක්‍රමය (මිශ්‍ර වී ඇත):

```
function checkout(price, quantity) {
  const total = price * quantity;
  console.log(total); // calculation එක ඇතුළේ side effect එකක්
  return total;
}
```

පිරිසිදු ක්‍රමය (වෙන් කර ඇත):

```
function getTotal(price, quantity) {
  return price * quantity; // Pure calculation එකයි
}

const total = getTotal(10, 3);
console.log(total); // Side effect එක පිටතින් සිදු කරයි
```

Part L: Callbacks and Higher-Order Functions

35. What Is a Callback?

Concept Explanation (Sinhala): Callback එකක් කියන්නේ තවත් function එකකට argument එකක් විදිහට අපි ඇතුළත් කරන function එකකටයි. ඒ කියන්නේ, "පස්සේ වෙලාවක run කරන්න" කියලා අපි වෙනත් function එකකට බාර දෙන function එකක්.

Code Block:

```
// Browser event එකේ callback එකුදාර පෙනෙන්නේ:
button.addEventListener("click", () => {
  console.log("Button clicked");
});
```

Code Breakdown (Sinhala):

- මෙතන තියන arrow function එක callback එකක්. ඒක කෙළින්ම run වෙන්නේ නැහැ. කවුරු හරි button එක click කරපු ගමන් තමයි ඒක ක්‍රියාත්මක වෙන්නේ.

36. Simple Callback Example

Concept Explanation (Sinhala): මෙහිදී අපි `runTask` කියන function එකට `sayHello` කියන function එක argument එකක් විදිහට පාස් කරන හැටි බලමු.

Code Block:

```
function runTask(callback) {
  callback(); // ලැබුණු callback function එක මෙතනදී run කරයි
}

function sayHello() {
  console.log("Hello");
}

runTask(sayHello); // නිවැරදි ක්‍රමය
// runTask(sayHello()); // වැරදි ක්‍රමයි! (වරහන් දැම්මොත් ඒක කලින්ම run වේ)
```

37. What Is a Higher-Order Function?

Concept Explanation (Sinhala): Higher-Order Function එකක් කියන්නේ පහත කරුණු දෙකෙන් අඩුම වශයෙන් එකක් හරි කරන function එකකටයි:

1. වෙනත් function එකක් argument එකක් විදිහට භාර ගන්නවා (callback) නම්, function.
2. තවත් අලුත් function එකක් නිර්මාණය කරලා එළියට දෙන (return කරන) function.

Code Block:

```
// 1. Function එකක් argument එකක් විදිහට ගන්නවා:
function runTask(callback) {
  callback();
}

// 2. Function එකක් return කරන එකක්:
function createGreeting() {
  return function() {
    return "Hello";
  };
}
```

Part M: Function Factories

38. What Is a Function Factory?

Concept Explanation (Sinhala): Function factory එකක් කියන්නේ සරලවම "වෙනත් විශේෂිත වූ functions හදන කර්මාන්තශාලාවක්" වගේ එකක්. මේකෙන් අපිට අවශ්‍ය විදිහට වැඩ කරන අලුත් function එකක් return කරනවා.

Code Block:

```
function multiplyBy(factor) {
  return function(number) {
    return number * factor;
  };
}

const double = multiplyBy(2); // 2න් ගුණ කරන function එකක් හදයි
const triple = multiplyBy(3); // 3න් ගුණ කරන function එකක් හදයි

console.log(double(5)); // Output: 10
console.log(triple(5)); // Output: 15
```

39. How the Function Factory Works

Concept Explanation (Sinhala):

- `const double = multiplyBy(2);` පේළියෙන් කරන්නේ `factor` එක `2` විදිහට මතක තියාගන්න අලුත් function එකක් හදන එකයි.
- ඊටපස්සේ `double(5)` call කළාම, ඒ හදපු function එක ඇතුළට `5` ගිහින් `5 * 2 = 10` ප්‍රතිඵලය දෙනවා.

Part N: Currying

40. What Is Currying?

Concept Explanation (Sinhala): Currying කියන්නේ එකපාර parameters ගොඩක් ගන්න function එකක් වෙනුවට, එක වරකට එක parameter එක බැගින් ගන්නා chain එකක් (වැල් පෙළක්) විදිහට ලිවීමේ ක්‍රමවේදයයි.

Code Block:

```
// සාමාන්‍ය ක්‍රමය:
function addNormal(a, b) {
  return a + b;
}

// Curried ක්‍රමය:
function addCurried(a) {
  return function(b) {
    return a + b;
  };
}

console.log(addCurried(2)(3)); // Output: 5
```

Code Breakdown (Sinhala):

- `addCurried(2)(3)` කියන කොටසින් වෙන්නේ: මුලින්ම `2` මතක තියාගන්නවා, දෙවන වරහනෙන් `3` එකතු කරනවා.

41. Beginner Currying Example

Concept Explanation (Sinhala): බදු ප්‍රතිශතය කලින්ම සෙට් කරලා තියාගෙන, පස්සේ වෙලාවක බඩු වල මිල විතරක් දීලා බදු ගණනය කරන හැටි බලමු.

Code Block:

```
function createTaxCalculator(taxRate) {
  return function(price) {
    return price * taxRate;
  };
}

const calculateTenPercentTax = createTaxCalculator(0.1);

console.log(calculateTenPercentTax(100)); // Output: 10
console.log(calculateTenPercentTax(50));  // Output: 5
```

Part O: Closures

42. What Is a Closure?

Concept Explanation (Sinhala): Closure එකක් කියන්නේ යම් කිසි inner function එකක්, තමන් නිර්මාණය වුණු මහා ෆංක්ෂන් එක (outer function එකෙන්) එළියට ගියත්, ඒ outer function එක ඇතුළේ තිබුණු variables තවදුරටත් මතක තබා ගැනීමේ හැකියාවටයි.

සරල සමීකරණය: `Closure = Function එක + එයාට මතක තියන පරිසර variables`

Code Block:

```
function createCounter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}

const counter = createCounter();

console.log(counter()); // Output: 1
console.log(counter()); // Output: 2
console.log(counter()); // Output: 3
```

43. How the Closure Works

Concept Explanation (Sinhala): සාමාන්‍යයෙන් function එකක් run වෙලා ඉවර වුණාම ඒක ඇතුළේ තිබුණු variables (උදාහරණ: `let count = 0`) මතකයෙන් මැකී යනවා. හැබැයි මෙතනදී `createCounter` එකෙන් එළියට දෙන inner function එක තවමත් `count` කියන variable එක පාවිච්චි කරනවා. ඒ නිසා JavaScript විසින් ඒ `count` variable එක මතකයේ රඳවා ගන්නවා. මේ රඳවා ගැනීම තමයි closure එකක් කියන්නේ.

44. Private State with Closures

Concept Explanation (Sinhala): Closures පාවිච්චි කරලා අපිට පිටස්තර අයට කෙළින්ම වෙනස් කරන්න බැරි රහසෙ තත්වයක් (private state) හදන්න පුළුවන්.

Code Block:

```
function createCounter() {
  let count = 0; // මෙය පිටතට රහසක්!
  return function() {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1

// console.log(count); // ERROR! පිටතින් access කළ නොහැක.
```

Code Breakdown (Sinhala):

- `count` variable එක තියෙන්නේ `createCounter` ඇතුළේ නිසා එළියෙන් ඒක කෙළින්ම වෙනස් කරන්න බැහැ. හැබැයි `counter()` function එක call කිරීමෙන් අපිට ඒක ආරක්ෂිතව update කරන්න පුළුවන්.

45. Closure Example: Secret Message

Concept Explanation (Sinhala): මුරපදයක් (password) නිවැරදිව ඇතුළත් කළහොත් පමණක් රහස් පණිවිඩය ලබා දෙන ආකාරය closures මඟින් නිර්මාණය කරමු.

Code Block:

```
function createSecret(secretPassword, secretMessage) {
  return function(passwordAttempt) {
    if (passwordAttempt === secretPassword) {
      return secretMessage;
    }
    return "Access denied";
  };
}

const getSecret = createSecret("js123", "Closures are useful");

console.log(getSecret("wrong")); // Output: Access denied
console.log(getSecret("js123")); // Output: Closures are useful
```

Part P: Core Sprint 5 Code Pattern

46. Core Code Pattern

Concept Explanation (Sinhala): මෙන්න මේ code එක ඔයාම තනියම type කරලා run කරලා බලන්න. Run කරන්න කලින් ලැබෙන output එක මොකක්ද කියලා හිතන් හිතන්න.

Code Block:

```
function greet(name = "Guest") {
  return `Hello, ${name}!`;
}

console.log(greet("Anna"));
console.log(greet());

const add = (a, b) => a + b;
console.log(add(2, 3));

function multiplyBy(factor) {
  return function (number) {
    return number * factor;
  };
}

const double = multiplyBy(2);
console.log(double(5));
```

Output:

```
Hello, Anna!
Hello, Guest!
5
10
```

Part Q: Practical Sprint 5 Functions

48. Function: cleanUsername(username)

Concept Explanation (Sinhala): පරිශීලකයා ඇතුළත් කරන messy (හිස්තැන් සහිත, අකුරු ලොකු කුඩා මිශ්‍ර) username පිරිසිදු කිරීමට මෙම function එක යොදාගනී.

Code Block:

```
function cleanUsername(username) {
  return username.trim().toLowerCase();
}

console.log(cleanUsername(" SANGEETH ")); // Output: sangeeth
console.log(cleanUsername(" ADA_LOVELACE ")); // Output: ada_lovelace
```

Code Breakdown (Sinhala):

- `trim()` මඟින් දෙපැත්තේ ඇති හිස්තැන් (spaces) ඉවත් කරයි.
- `toLowerCase()` මඟින් සියලුම අකුරු කුඩා අකුරු (lowercase) බවට පත් කරයි.
- මෙම ක්‍රම දෙක එකට ලිවීම method chaining ලෙස හඳුන්වයි.

49. Function: formatFullName(firstName, lastName)

Concept Explanation (Sinhala): නම් දෙකක් වෙන වෙනම ගෙන, ඒවායේ හිස්තැන් අයින් කර, එකතු කර පිරිසිදු සම්පූර්ණ නමක් සාදයි.

Code Block:

```
function formatFullName(firstName, lastName) {
  const cleanFirst = firstName.trim();
  const cleanLast = lastName.trim();
  return `${cleanFirst} ${cleanLast}`;
}

console.log(formatFullName(" Ada", "Lovelace ")); // Output: Ada Lovelace
```

52. Function: calculateGrade(score)

Concept Explanation (Sinhala): ලබා ගන්නා ලකුණු ප්‍රමාණය අනුව ශ්‍රේණිය (Grade) ගණනය කරයි. මෙහිදී `return` එකක් සිදු වූ සැනින් function එක නවතින නිසා අනවශ්‍ය `else` භාවිතය මඟ හැරේ.

Code Block:

```
function calculateGrade(score) {
  if (score >= 90) {
    return "A";
  }
  if (score >= 80) {
    return "B";
  }
  if (score >= 70) {
    return "C";
  }
  return "Keep practicing";
}

console.log(calculateGrade(95)); // Output: A
console.log(calculateGrade(82)); // Output: B
console.log(calculateGrade(65)); // Output: Keep practicing
```

Part R: Mini-Build 1 - Shopping Cart Functions

53. Task & Implementation

Concept Explanation (Sinhala): හැම වෙලාවෙම එකම ලොකු function එකක් ඇතුළේ හැමදේම ලියන්නේ නැතුව, කුඩා කාර්යයන් සඳහා වෙන වෙනම සරල functions ලියා ඒවා එකිනෙක සම්බන්ධ කරන ආකාරය මෙයින් ඉගෙන ගත හැක.

Code Block:

```
function getSubtotal(price, quantity) {
  return price * quantity;
}

function getTax(subtotal, taxRate) {
  return subtotal * taxRate;
}

function getFinalTotal(subtotal, tax) {
  return subtotal + tax;
}

const subtotal = getSubtotal(20, 3);
const tax = getTax(subtotal, 0.1);
const finalTotal = getFinalTotal(subtotal, tax);

console.log(subtotal);    // Output: 60
console.log(tax);         // Output: 6
console.log(finalTotal); // Output: 66
```

Code Breakdown (Sinhala):

1. `getSubtotal` එකෙන් මුළු භාණ්ඩ වල වටිනාකම හදලා දෙනවා.
2. `getTax` එකෙන් ඒ ගාණට අදාළ බදු මුදල හදනවා.
3. `getFinalTotal` එකෙන් ඒ දෙකම එකතු කරලා අවසාන ගාණ ලබා දෙනවා.

Part V: Debug Challenges

67. Debug Challenge 1: Missing Return

Concept Explanation (Sinhala): ආධුනිකයන් අතින් සිදුවන ප්‍රධානතම වැරැද්දක් නම් අගයක් ගණනය කළත් එය `return` කිරීමට අමතක වීමයි. එවිට output එක ලෙස ලැබෙන්නේ `undefined` ය.

Code Block: වැරදි code එක:

```
function add(a, b) {
  a + b; // මෙතන return ලියා නැත
}
console.log(add(2, 3)); // Output: undefined
```

නිවැරදි කළ code එක:

```
function add(a, b) {
  return a + b;
}
console.log(add(2, 3)); // Output: 5
```

70. Debug Challenge 4: Calling a Callback Too Early

Concept Explanation (Sinhala): Callback එකක් තවත් function එකකට පාස් කිරීමේදී වරහන් `()` රහිතව function එකේ නම පමණක් ලිවිය යුතුය. වරහන් දැමුවහොත් එය වහාම run වී එහි ප්‍රතිඵලය පාස් වේ.

Code Block: වැරදි ක්‍රමය:

```
runTask(sayHello()); // sayHello වහාම run වේ
```

නිවැරදි ක්‍රමය:

```
runTask(sayHello); // runTask එක ඇතුළේදී පසුව run වීමට ලැබෙයි
```

Part W: Common Sprint 5 Mistakes

Summary of Mistakes to Avoid (සිංහල මතක් කිරීම්):

1. **Forgetting to Call the Function:** `greet` ලෙස පමණක් ලිවීමෙන් වැඩක් නැත. Function එක run කිරීමට නම් `greet()` ලෙස වරහන් සහිතව ලිවිය යුතුය.
2. **Forgetting return:** Function එකකින් අගයක් එළියට ගැනීමට නම් අනිවාර්යයෙන්ම `return` keyword එක භාවිත කරන්න.
3. **Thinking console.log() is return:** `console.log` එකෙන් කරන්නේ print කිරීම පමණි. එයින් ලැබෙන අගය නැවත ගණනය කිරීම් වලට පාවිච්චි කළ නොහැක.
4. **Using Local Variable Outside Scope:** Function එකක් හෝ Block එකක් ඇතුළත හදන `let` / `const` variables එළියෙන් පාවිච්චි කළ නොහැක.

Final Sprint 5 Summary (අවසාන සාරාංශය)

- **Function:** නැවත නැවත භාවිත කළ හැකි code සමූහයකි.
- **Parameters & Arguments:** Parameters යනු placeholders වන අතර Arguments යනු සැබෑ වටිනාකම් වේ.
- **Return:** අගයක් පිටතට ලබා දෙන අතර එතනින් function එක නතර වේ.
- **Scope:** Variables පාවිච්චි කළ හැකි සීමාව පාලනය කරයි.
- **Pure Functions:** එකම input එකට එකම output එකක් ලබා දෙන, බාහිර දේ වෙනස් නොකරනPredictable functions වේ.
- **Closure:** තමන් හැදුණු outer scope එකේ variables මතක තබා ගන්නා inner function එකකි. මෙය private state සෑදීමට උපකාරී වේ.